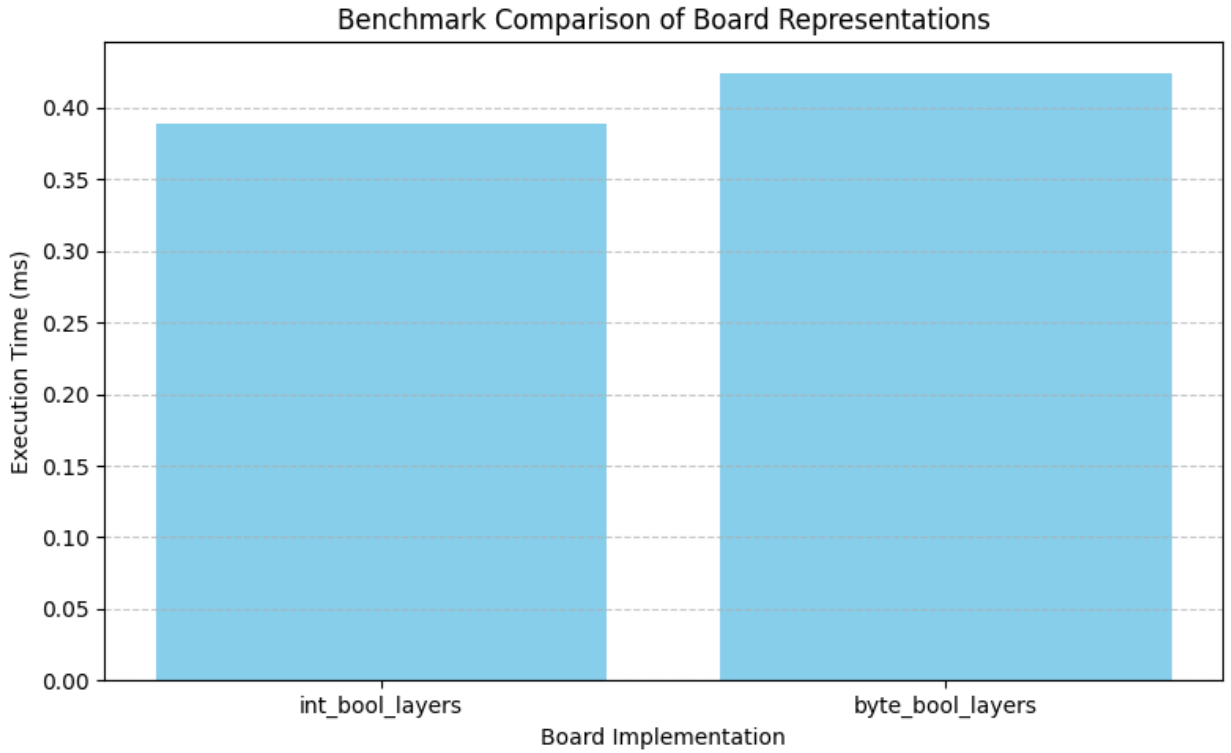
Assignment 2

Group 7: Fina, David, Jenrik, Jesper

Java Benchmarks

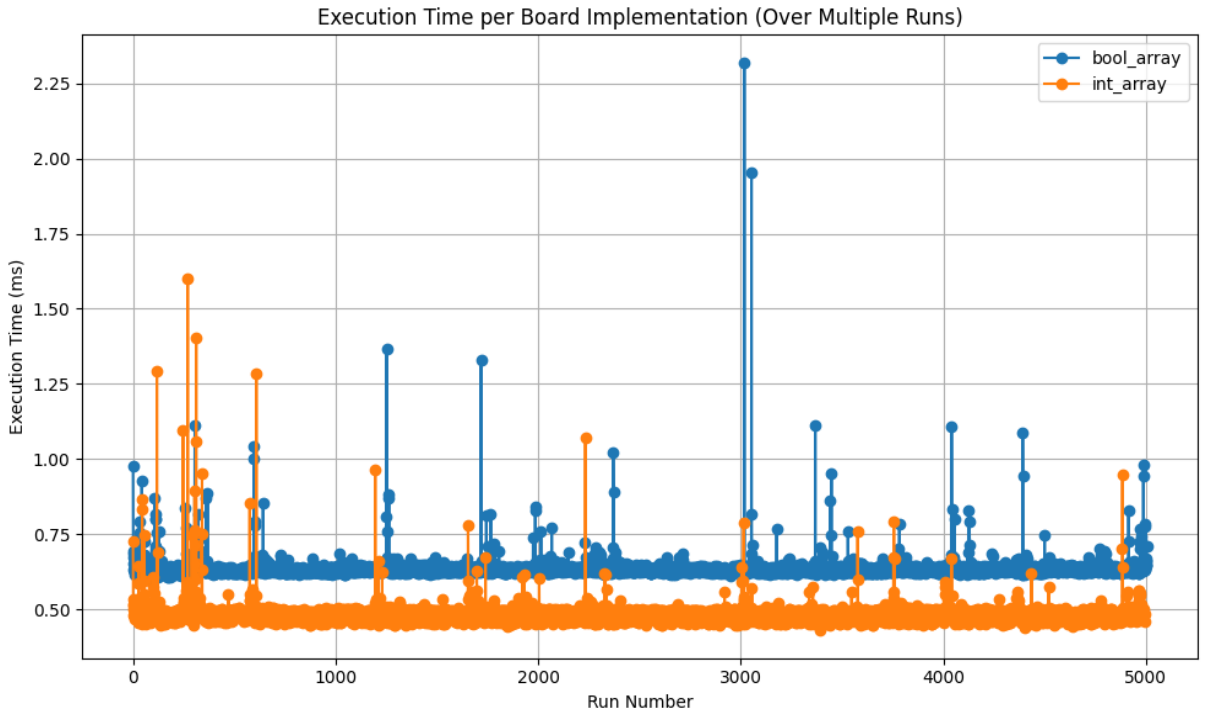
1. byte vs. int
2. int[][] vs bool[][][] (with layers)

Results byte vs int



=> using int is faster

Results int[][] vs bool[][]



=> using int[][] is faster

Java Benchmark learnig

- Optimising without testing does not help
- It is not obvious what is faster.
- So only use the complicated approach if it's really faster.

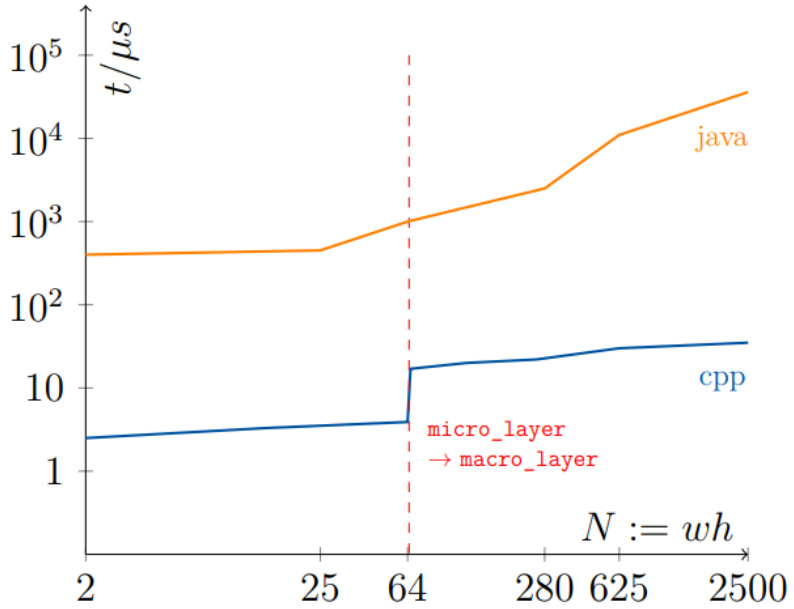
Finding Successor States

- Kogge Stone algorithm
- LUT for Edge wrap masks
- Recursion for transitions
- Color map for handling inversion and choice

Finding Successor States

Complexity: O (T log (max { w, h }))

100 – 1000x faster than Java



Full Gameplay Implementation

- **Client** holds instance member and _me
- **Instance** provides functions to access game state & map and to apply Moves
- **Move** encodes a server message describing a move performed by an arbitrary player

Task 3: evaluate_domination()

- Returns: $100 \times \frac{\text{tiles of the player}}{\text{all occupied tiles}}$

- Best-Case = 100
- Worst-Case = 0

Task 3: evaluate_mobility()

- Returns:

available moves for the player – average available moves of enemies

- Best-Case: +∞
- Worst-Case: -∞

Task 3: evaluate_edges()

- Create a Bit-Mask, with 1s for all tiles neighboring the players tiles
- Returns: the Bit Sum of Bit-Mask & (Holes | Board_edge)

- Best-Case: +∞
- Worst-Case: 0

Task 3: evaluate_neighbours()

- Create a Bit-Mask, with 1s for all tiles neighboring the players tiles
- Returns: the Bit Sum of Bit-Mask & all_enemy_layers

- Assumed Best-Case: Large but not too large (uncertain so far)
- Assumed Worst-Case: 0 or ∞

Task 3: evaluate_special()

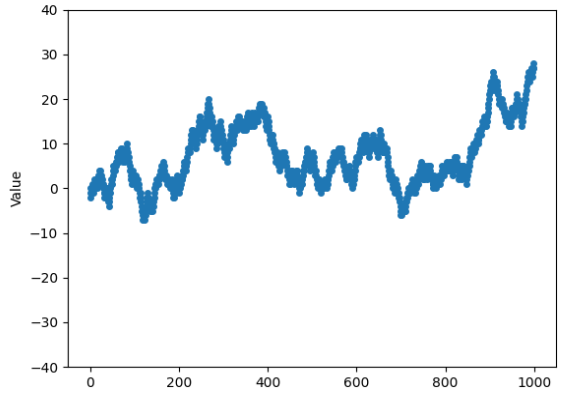
- For every enemy we calculate:
  - Result += choice tiles a enemy can pick * min(enemy_tiles – player_tiles, 0)
  - Result += inversion tiles a enemy can pick * inversionpartner_tiles – player_tiles
- Returns: Result

- Best-Case: ∞
- Worst-Case: -∞

Task 3: The game state value

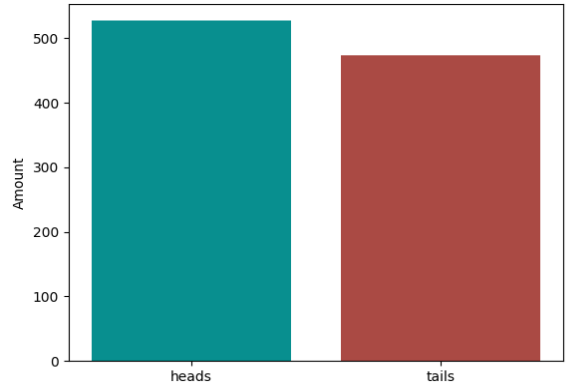
Evaluation = a * evaluate_domination()
+ b * evaluate_mobility()
+ c * evaluate_edges()
+ d * evaluate_neighbours()
+ e * evaluate_special()
+ f * evaluate_override_stones()

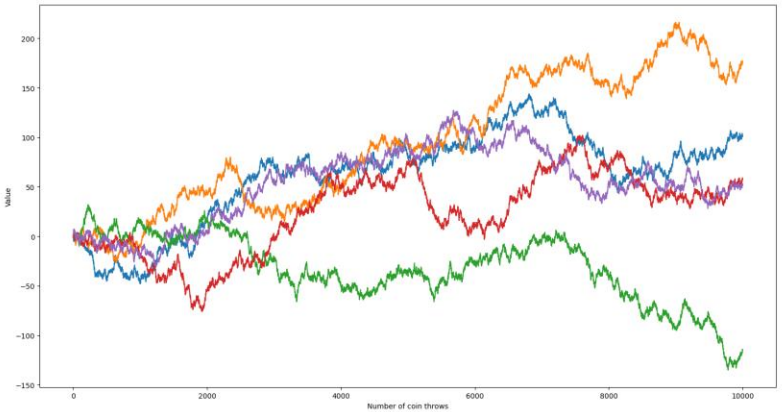
Task 4: Plotting



Tracked parameters:

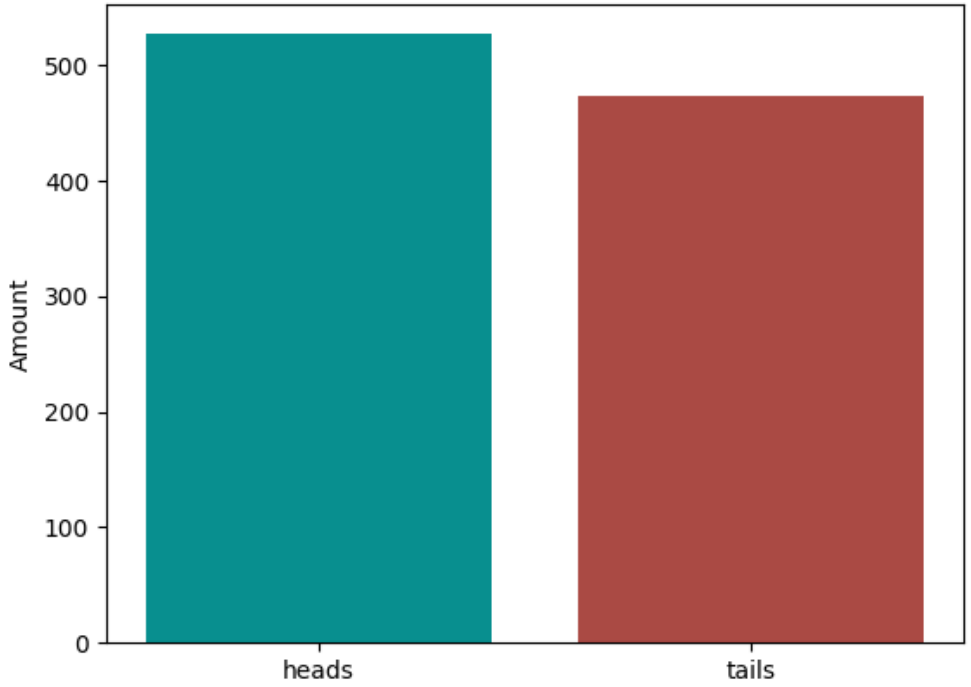
1. Total amount of heads & tails
2. Maximum & minimum value
3. Longest streak of heads & tails
4. Iterations/coin throws reaching 0



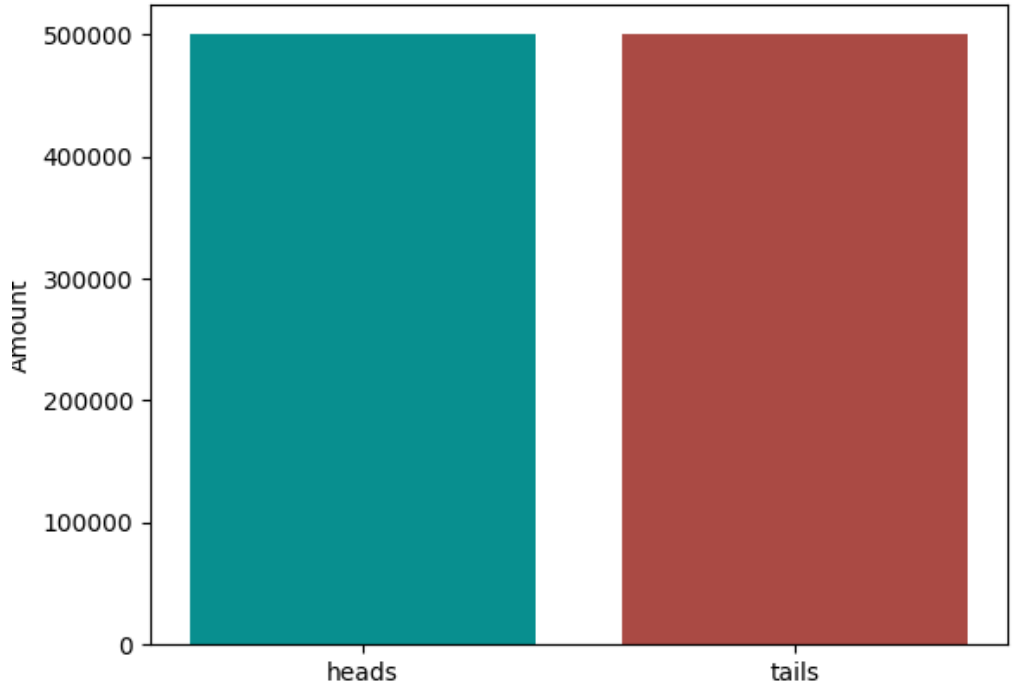


Task 4: Plotting

'Law of large numbers'



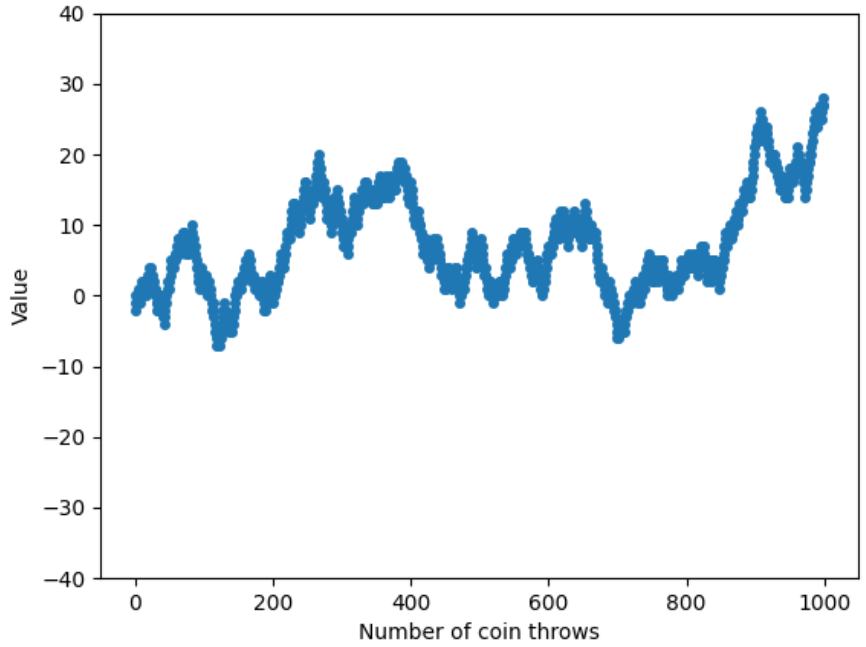
Heads = 572, tails = 473; Sample Size = 1,000



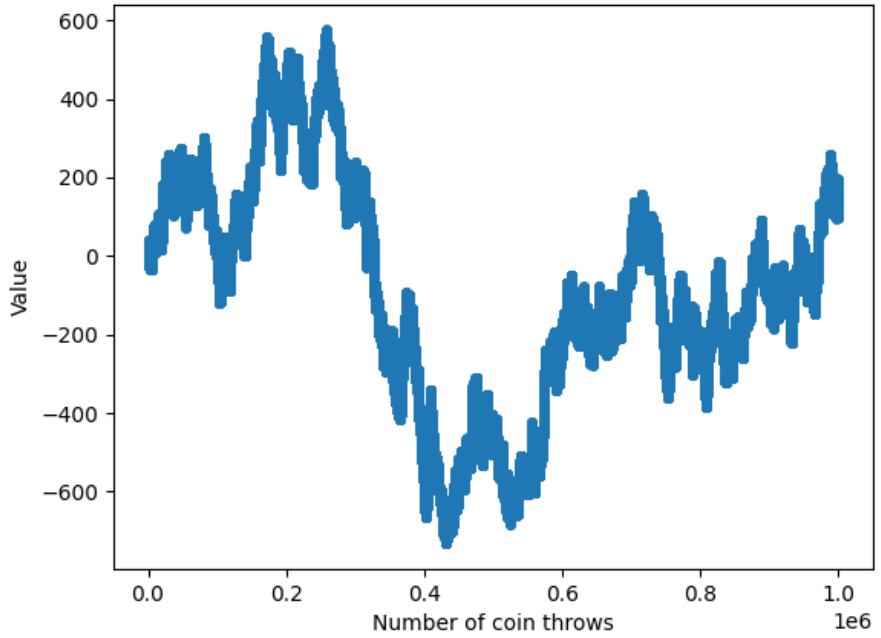
Heads = 49981, tails = 50019; Sample Size = 1,000,000

Task 4: Plotting

'Law of large numbers'



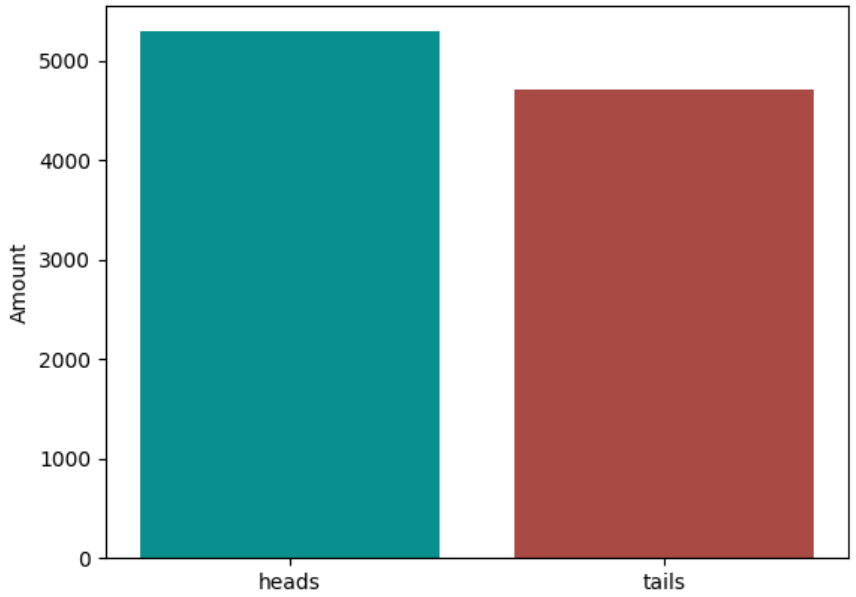
Sample Size = 1,000



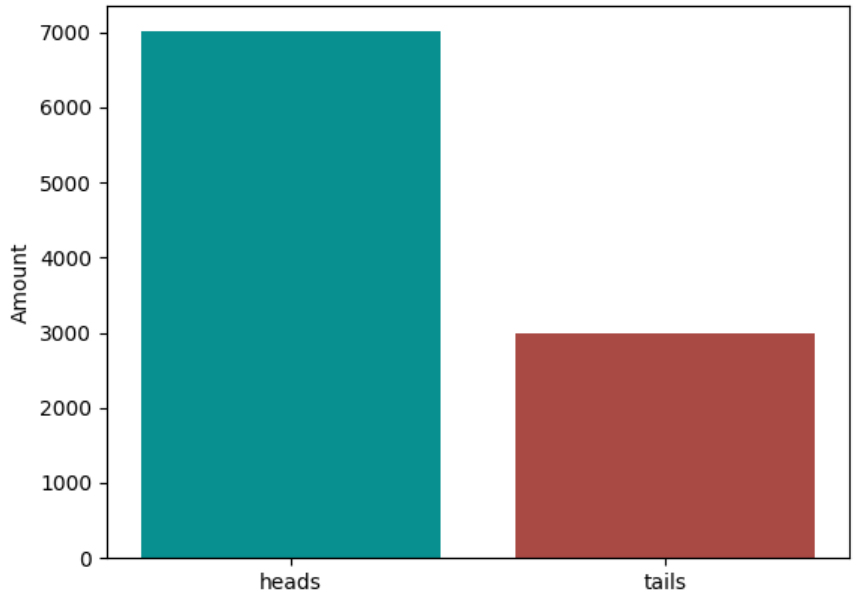
Sample Size = 1,000,000

Task 4: Plotting

Weighted Coin



Prob: heads = 0.51, tails = 0.49
Sample Size = 10,000



Prob: heads = 0.6, tails = 0.4
Sample Size = 1,000,000